HELEP

Humanitarian Emergency Location & Emergency Protocol

Demo Presentation Script

SEN3244 Software Architecture — Engr. Tekoh Palma

5-Minute Screen Recording — Part I (10 marks)

Demo Requirements Checklist

#	Spec requirement (Part I)	Stage in this script	Time
1	kubectl get all -n helep + PV/PVC	Stage 2	0:00–0:50
2	Trigger SOS via curl + tail notification logs	Stage 3	0:50–2:00
3	Cancel SOS — show responder released	Stage 4	2:00–2:45
4	Chaos: delete dispatch pod, prove saga survives	Stage 5	2:45–3:30
5	Grafana dashboard with live non-zero metrics	Stage 6	3:30–4:15
6	CI/CD pipeline final green run	Stage 7	4:15–5:00

■■ Read this script once in full before recording. Do a dry run first. The whole demo must fit within 5 minutes.

HOW TO READ THIS SCRIPT

Symbol	Meaning
"Quoted text"	Read this out loud — word for word or close to it
`command --here`	Type and run this in your terminal
■■ **Warning text**	Pay attention — easy mistake point
■ Tip text	Helpful hint to make the demo smoother
■ **Action**	Do this on screen without speaking

■■ **Before you start recording: make sure your cluster is running, all pods are Ready, Grafana is open in a browser tab, and your Jenkins pipeline has already completed at least one green run. Screenshot or screen-record the green run before the 5-minute clock starts.**

■ Split your screen: terminal on the left, browser (Grafana / Jenkins) on the right. This saves time switching windows during the demo.

STAGE 1 | INTRODUCTION — WHO YOU ARE AND WHAT HELEP DOES | sets context, not directly graded

Approx. time: ~0:00 – 0:40

Start recording. Open with your face or voice on camera. Speak clearly and at a steady pace.

"Good afternoon. My name is [say your full name], matricule [say your matricule number], studying Software Architecture with Engr. Tekoh Palma."

"In this video I will demonstrate my Kubernetes deployment of HELEP — the Humanitarian Emergency Location and Emergency Protocol platform."

Briefly describe the system — keep this under 20 seconds:

"HELEP is a real-time emergency dispatch system built for Cameroon. When a citizen is in danger they fire an SOS from their phone. The system picks the nearest free responder, notifies them, and feeds a police analytics dashboard — all through an Apache Kafka event bus. Five FastAPI micro-services handle identity, SOS triggering, dispatch, notifications, and analytics. The services are containerised with Docker, deployed on Kubernetes using a Helm umbrella chart, and the Kafka cluster is managed by the Strimzi operator."

"I will now demonstrate each component in turn, following the demo requirements from the project specification."

■ Keep this introduction tight — you only have 5 minutes total. Do not spend more than 40 seconds here.

STAGE 2 | CLUSTER STATE — KUBECTL GET ALL + PV/PVC | Spec requirement 1 of 6

Approx. time: ~0:40 – 1:30

Switch to your terminal. This stage proves the full platform is running on Kubernetes with persistent storage.

"Let me start by showing the cluster state. Everything is deployed in the helep namespace."

■ ***Type this command:***

```
kubectl get all -n helep
```

"You can see all five service Deployments — user-service, sos-service, dispatch-service, notification-service, and analytics-service — each with its pods in the Running state."

"You can also see the HorizontalPodAutoscalers, ClusterIP Services, and the Kafka bootstrap service managed by Strimzi."

■ ***Now show persistent storage:***

```
kubectl get pv,pvc -n helep
```

"Each service has its own PersistentVolumeClaim — one per service, mounted at /data inside the container. This is where the SQLite database files live. If a pod restarts, the data survives because it is on the persistent volume, not inside the container layer."

■ ***Optionally show the Strimzi Kafka pods:***

```
kubectl get pods -n kafka
```

"The Kafka broker is running in its own kafka namespace, managed by the Strimzi operator. The HELEP services connect to it via the bootstrap service address configured in the Helm values file."

■■ **If any pod shows CrashLoopBackOff or Pending, stop the recording and fix it before continuing. The spec requires all pods to be Running.**

■ Run 'kubectl get pods -n helep' first to confirm everything is Ready before you start the clock on your recording.

STAGE 3 | LIVE SOS SAGA — TRIGGER VIA CURL + TAIL LOGS | Spec requirement 2 of 6

Approx. time: ~1:30 – 2:30

This is the centrepiece of the demo. You will prove the full choreographed saga works end-to-end: citizen triggers SOS → dispatch assigns responder → notification service fires. Use two terminal panels side by side if possible.

Step 3a — Open a log tail in one terminal panel:

"I am going to open a live log stream from the notification service so we can watch it receive the assignment event in real time."

```
kubectl logs -f deployment/notification-service -n helep
```

Step 3b — Port-forward user-service and sos-service in a second panel:

```
kubectl port-forward svc/user-service 8001:8001 -n helep &
kubectl port-forward svc/sos-service 8002:8002 -n helep &
```

Step 3c — Register a citizen account:

"First I will register a citizen account. I am using curl to call the user-service signup endpoint."

```
curl -s -X POST http://localhost:8001/signup \
-H "Content-Type: application/json" \
-d '{"phone":"+237600000099","password":"demo1234","role":"citizen"}' \
| python3 -m json.tool
```

"The service returns a JWT token. I will save that token in a variable for the next call."

```
TOKEN=$(curl -s -X POST http://localhost:8001/login \
-H "Content-Type: application/json" \
-d '{"phone":"+237600000099","password":"demo1234"}' \
| python3 -c "import sys,json; print(json.load(sys.stdin)['token'])")
echo "Token obtained: ${TOKEN:0:40}..."
```

Step 3d — Fire the SOS:

"Now I fire the SOS. I am passing GPS coordinates that place the citizen in Douala, Cameroon."

```
curl -s -X POST http://localhost:8002/sos \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"lat":4.0500,"lon":9.7700,"mode":"online"}' \
| python3 -m json.tool
```

"The sos-service created an incident and published the sos.triggered event to Kafka with the incident ID as the partition key. Watch the notification-service log on the left — you should see it receive the responder.assigned event within less than a second."

■ ***Point at the notification-service log output and read the key line:***

“There — the notification service consumed the responder.assigned event and logged 'notification.delivered'. The full saga — from citizen trigger to responder notification — completed in under one second, which satisfies the performance requirement in the SRS.”

```
INCIDENT_ID=
```

■ Save the incident_id from the curl output — you will need it in Stage 4 to cancel the SOS.

■■ **If the notification log shows nothing after 5 seconds, check that dispatch-service is Running and that the KAFKA_BOOTSTRAP address in your Helm values matches the Strimzi service name.**

STAGE 4 | SOS CANCELLATION — RESPONDER RELEASED | Spec requirement 3 of 6

Approx. time: ~2:30 – 3:00

You will cancel the SOS you just created and prove that the assigned responder is freed — the core rollback step of the choreographed saga.

“Now I will demonstrate the cancellation path — the compensating transaction in the choreographed saga.”

Step 4a — Cancel the SOS:

```
curl -s -X POST http://localhost:8002/sos/$INCIDENT_ID/cancel \
-H "Authorization: Bearer $TOKEN" \
| python3 -m json.tool
```

“The sos-service set the incident status to CANCELLED and published the sos.cancelled event. Watch the dispatch-service logs — it should consume that event and release the responder's busy flag.”

Step 4b — Confirm the release in dispatch-service logs:

```
kubectl logs deployment/dispatch-service -n helep --tail=20
```

“You can see the log line 'dispatch.released' with the incident ID. The responder is now free to be assigned to a new SOS. This is the compensation step of the saga — the system rolled back the reservation cleanly without any manual intervention and without a central orchestrator.”

■ If you want to be thorough, fire a second SOS immediately after cancelling — the same responder should be assigned again, proving the busy flag was reset correctly.

STAGE 5 | CHAOS TEST — DELETE DISPATCH-SERVICE POD | Spec requirement 4 of 6

Approx. time: ~3:00 – 3:45

This is the most impressive part of the demo. You will kill the dispatch pod mid-saga and prove that Kafka re-delivers the uncommitted offset to the new pod, so no SOS is dropped. This directly demonstrates at-least-once delivery and consumer group rebalancing.

"Now for the chaos test. I am going to delete the dispatch-service pod while Kafka has an unprocessed event, and prove the saga still completes."

Step 5a — Fire a new SOS (do not cancel it):

```
curl -s -X POST http://localhost:8002/sos \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"lat":4.0611,"lon":9.7779,"mode":"online"}' \
| python3 -m json.tool
```

Step 5b — Immediately delete the dispatch-service pod:

"I am now deleting the dispatch-service pod to simulate a crash."

```
kubectl delete pod -l app.kubernetes.io/name=dispatch-service -n helep
```

"Kubernetes detects the pod is gone and immediately starts a replacement. In the meantime, the sos.triggered event that was just published to Kafka has not been acknowledged — the offset was not committed because the handler had not finished."

Step 5c — Watch the new pod start:

```
kubectl get pods -n helep -w
```

"The new dispatch-service pod is starting. Because we use Kafka consumer groups with manual offset commit, the event is still sitting at the last committed offset. When the new pod joins the consumer group, Kafka triggers a rebalance and reassigns the partition to it. The new pod will re-consume and process the event."

Step 5d — Confirm the saga completed:

```
kubectl logs deployment/dispatch-service -n helep --tail=20
kubectl logs deployment/notification-service -n helep --tail=10
```

"You can see 'dispatch.assigned' in the new pod's logs, and 'notification.delivered' in the notification logs. The SOS was successfully dispatched even though the dispatch service crashed and restarted mid-saga. This is exactly what at-least-once Kafka delivery gives us — no event is permanently lost."

■■ The chaos test assumes the sos.triggered event was published before the pod died — which it was, because the SOS endpoint writes to the DB and publishes to Kafka before returning the HTTP response. If the examiner asks why this works, explain the outbox-lite pattern.

■ Practise the timing of this step. Delete the pod within 1-2 seconds of firing the SOS to make the rebalance visible on screen.

STAGE 6 | GRAFANA DASHBOARD — LIVE NON-ZERO METRICS | Spec requirement 5 of 6

Approx. time: ~3:45 – 4:20

Switch to your browser where Grafana is already open. The dashboard should show live data from all the curl calls you made in Stages 3–5.

"Now let me show the observability stack. I have Prometheus and Grafana deployed in the observability namespace, scraping metrics from all five HELEP services."

■ ***Click on the HELEP Platform Overview dashboard:***

"This is the Grafana dashboard I provisioned as part of the deployment. It is defined in dashboards/helep-overview.json and loaded automatically when Grafana starts."

■ ***Point to each panel and say:***

"The top-left panel shows SOS triggers per minute — you can see the spike from the curl commands I just ran."

"The top-right panel shows dispatch assignment counts over time — non-zero after the saga we ran."

"The bottom-left table shows the circuit breaker state for each service — all currently CLOSED, meaning Kafka is healthy."

"The bottom-right panel shows Kafka consumer group lag — currently at or near zero, confirming all events have been processed."

■ ***Optional — show Prometheus targets page:***

"I can also show the Prometheus targets page to confirm all five service endpoints are being scraped successfully."

```
# In browser: http://localhost:9090/targets
```

"Every service shows State: UP, confirming the Prometheus annotations on the pods are working correctly."

■ If your dashboard panels show 'No data', run two or three more curl SOS triggers before recording this stage. Prometheus scrapes every 15 seconds, so there may be a short lag.

■■ **Make sure you port-forward Grafana before recording: kubectl port-forward svc/grafana 3000:3000 -n observability**

STAGE 7 | CI/CD PIPELINE — JENKINS GREEN RUN | Spec requirement 6 of 6

Approx. time: ~4:20 – 5:00

Switch to your Jenkins browser tab. You should already have a completed green pipeline run from before you started recording. Walk through each stage briefly.

"Finally, let me show the CI/CD pipeline. I am using Jenkins with a declarative Jenkinsfile — GitHub Actions is not accepted for this exercise."

■ ***Open the completed green pipeline build:***

"This is the Jenkinsfile I wrote in ci/Jenkinsfile. It has seven stages that run in sequence."

■ ***Click through each stage and narrate:***

"Stage one: Lint. It runs ruff check on all five service directories and fails the build if any Python style violations are found."

"Stage two: Build. Docker builds each of the five service images using the multi-stage Dockerfiles. Each image is based on python:3.12-slim, runs as a non-root user, and exposes only its specific port."

"Stage three: Security Scan. Trivy scans every image for HIGH and CRITICAL CVEs and fails the pipeline if any are found. This enforces the DevSecOps requirement from the specification."

"Stage four: Push. The tagged images are pushed to my Docker Hub namespace."

"Stage five: Deploy. Helm upgrade installs the umbrella chart to the helep namespace, applying all Deployments, Services, HPAs, PVCs, ConfigMaps, Secrets, and the Ingress resource."

"Stage six: Smoke Test. The pipeline port-forwards each service and hits /healthz to confirm all five are responding. It also runs a quick end-to-end saga — signup, then SOS trigger — and checks the analytics endpoint for a non-zero event count."

"Stage seven: Production gate. This is a manual approval step — a Jenkins input() — that pauses the pipeline and waits for an authorised user to click Approve before the production deploy proceeds. This is the DevSecOps change-control requirement."

"All seven stages completed successfully — the build is green. This confirms the entire platform can be built, scanned, and deployed from a clean state in a single pipeline run."

Closing Summary

"To summarise what I have demonstrated in this video:"

- A full Kubernetes deployment of HELEP with all five services Running, persistent storage via PVCs, and Kafka managed by Strimzi.
- A live end-to-end SOS saga — from citizen trigger through dispatch assignment to responder notification — completing in under one second.
- The cancellation compensation path — the sos.cancelled event releasing the responder cleanly.
- A chaos test proving that Kafka at-least-once delivery and consumer group rebalancing ensure no SOS is dropped when a pod crashes mid-saga.
- A Grafana dashboard showing live non-zero metrics from Prometheus.
- A complete green Jenkins CI/CD pipeline with lint, build, Trivy scan, push, deploy, smoke test, and manual production gate.

"Thank you for watching. My name is [your name], matricule [your number]. All source code, manifests, Helm charts, and documentation are in the submitted zip file named [your-matricule]-[your-name]-helep.zip."

Quick Reference — Commands You Will Type During the Demo

```
# Stage 2 — Cluster state
kubectl get all -n helep
kubectl get pv,pvc -n helep
kubectl get pods -n kafka
```

```
# Stage 3 — Port forwards
kubectl port-forward svc/user-service 8001:8001 -n helep &
kubectl port-forward svc/sos-service 8002:8002 -n helep &
kubectl logs -f deployment/notification-service -n helep
```

```
# Stage 3 — Signup + login + SOS trigger
curl -s -X POST http://localhost:8001/signup \
-H "Content-Type: application/json" \
-d '{"phone":"+237600000099","password":"demo1234","role":"citizen"}'
TOKEN=$(curl -s -X POST http://localhost:8001/login \
-H "Content-Type: application/json" \
-d '{"phone":"+237600000099","password":"demo1234"}' \
| python3 -c "import sys,json; print(json.load(sys.stdin)['token'])")
curl -s -X POST http://localhost:8002/sos \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"lat":4.0500,"lon":9.7700,"mode":"online"}' | python3 -m json.tool
```

```
# Stage 4 — Cancel SOS
curl -s -X POST http://localhost:8002/sos/$INCIDENT_ID/cancel \
```

```
-H "Authorization: Bearer $TOKEN" | python3 -m json.tool
kubectl logs deployment/dispatch-service -n helep --tail=20
```

```
# Stage 5 — Chaos
kubectl delete pod -l app.kubernetes.io/name=dispatch-service -n helep
kubectl get pods -n helep -w
kubectl logs deployment/dispatch-service -n helep --tail=20
```

```
# Grafana port-forward (run before recording)
kubectl port-forward svc/grafana 3000:3000 -n observability &
```

Pre-Recording Checklist

- ■ All 5 HELEP pods show Running/Ready — kubectl get pods -n helep
- ■ Kafka broker pod is Running — kubectl get pods -n kafka
- ■ Grafana is accessible at localhost:3000 with a provisioned dashboard
- ■ Jenkins shows at least one completed green build
- ■ You have the incident_id variable ready or know where to copy it from
- ■ Terminal font size is large enough to read in the recording
- ■ Screen resolution is at least 1920x1080 for the recording
- ■ You have done one full dry run of the entire script